



Instituto Politécnico do Cávado e do Ave

Engenharia de Sistemas Informáticos

Estruturas de Dados Avançadas – Fase 1

Listas Ligadas

Relatório de Projeto

Rúben Oliveira – Aluno Nº 24861

Março de 2025

Estrutura do Relatório

O relatório está organizado da seguinte forma:

1. [Introdução](#)
2. [Apresentação do Problema](#)
3. [Regras Definidas](#)
4. [Estrutura do Projeto](#)
5. [Estrutura de Dados Utilizada](#)
6. [Funcionalidades Implementadas](#)
7. [Casos de Teste](#)
8. [Conclusão](#)

1. Introdução

O presente trabalho foi desenvolvido no âmbito da unidade curricular de **Estruturas de Dados Avançadas**, inserida no curso de Engenharia de Sistemas Informáticos. O principal objetivo deste projeto é a aplicação prática de estruturas de dados dinâmicas, em particular listas ligadas, para resolver um problema computacional com múltiplos elementos, relações e operações sobre uma estrutura que evolui ao longo do tempo.

2. Apresentação do Problema

O problema proposto consiste no desenvolvimento de um sistema para representar uma rede de antenas de transmissão, onde cada antena possui uma frequência e uma localização definida por coordenadas num plano bidimensional.

A estrutura de dados deve permitir a gestão dinâmica dessa rede, com funcionalidades como inserção, remoção e análise de possíveis interferências entre antenas. Para além das antenas, o sistema deverá ser

capaz de representar zonas afetadas por interferência, de acordo com critérios previamente definidos.

O sistema deve ainda ser capaz de carregar e guardar a informação da rede num ficheiro de texto, permitindo o armazenamento de diferentes versões ou estados da estrutura ao longo do tempo.

3. Regras Definidas

Durante o desenvolvimento deste projeto, foi necessário estabelecer um conjunto de regras e restrições adicionais que não estavam explícitas no enunciado original, com o objetivo de tornar o comportamento da estrutura mais consistente e realista face ao problema modelado.

As principais regras definidas foram as seguintes:

- **Frequência case-insensitive:** As frequências das antenas são tratadas de forma insensível a maiúsculas e minúsculas. Por exemplo, uma antena com frequência 'A' será considerada igual a uma com frequência 'a'.
- **Carácter reservado '#':** O carácter '#' está reservado para representar zonas de interferência ou efeitos nefastos, e não pode ser utilizado como frequência de uma antena.
- **Proibição de coordenadas duplicadas:** Não é permitida a existência de duas antenas na mesma posição (coordenadas x e y). No entanto, uma posição já ocupada por uma antena pode conter adicionalmente um efeito nefasto, caso este ocorra.
- **Distância mínima entre antenas da mesma frequência:** Foi definido um limite mínimo de distância entre antenas com a mesma frequência. Se duas antenas violarem essa distância mínima, considera-se que existe interferência, sendo gerado um efeito nefasto entre elas.
- **Área afetada pelo efeito nefasto:** Quando ocorre interferência, é registado um conjunto de posições afetadas ao redor da zona central do conflito. A extensão dessa área pode variar consoante a distância entre as antenas em conflito.
- **Modularização e separação de responsabilidades:** O projeto foi organizado em múltiplos módulos (`ed_io`, `ed_validator`, `ed_effects`, etc.), sendo utilizadas funções auxiliares `static` para validações internas, garantindo encapsulamento e reutilização de código.

4. Estrutura do Projeto e Arquitetura

4.1 Convenções e Estilo de Código

Durante o desenvolvimento do projeto, optei por adotar o estilo de nomeação `snake_case` para funções e variáveis, em detrimento do `CamelCase`. Esta escolha prende-se com o facto de `snake_case` ser amplamente utilizado em projetos escritos em linguagem C, por oposição a `CamelCase`, mais comum em linguagens orientadas a objetos como Java ou C#.

Algumas convenções utilizadas:

- **Funções e variáveis:** `snake_case`, ex: `insert_ed`, `load_from_file`
- **Estruturas e tipos personalizados:** `CamelCase`, ex: `ED`, `Dimensions`
- **Constantes:** Maiúsculas com underscore, ex: `MAX_ROWS`, `MIN_DISTANCE`
- **Ficheiros .c e .h:** nomes em minúsculas com underscore, ex: `ed_utils.c`, `ed_io.h`

Esta padronização melhora a legibilidade do código, facilita a navegação entre funções e permite distinguir visualmente os diferentes tipos de elementos presentes no projeto.

Embora o professor recomende o uso de **CamelCase**, considere importante manter coerência com práticas mais comuns na comunidade C, especialmente em projetos que seguem filosofia de baixo nível e maior proximidade ao sistema.

5.2 Organização Modular

O projeto foi desenvolvido com uma abordagem modular, de forma a garantir a organização, manutenibilidade e reutilização do código. Seguindo princípios de **Clean Code**, as funções foram nomeadas de forma clara, os ficheiros foram separados conforme as suas responsabilidades, e a lógica foi dividida por módulos bem definidos.

A estrutura de diretórios segue uma organização clássica para projetos em C:

```
|— src/           # Código-fonte principal (implementações .c)
|— include/       # Ficheiros de cabeçalho (.h)
|— input/         # Exemplos de ficheiros de entrada
|— docs/         # Documentação e relatório
|— bin/          # Binários e ficheiros gerados
|— Doxyfile       # Configuração do Doxygen
|— README.md     # Guia de utilização e explicação do projeto
```

Módulos e Responsabilidades

- **main.c** – Ponto de entrada da aplicação. Inicializa o programa, apresenta o menu e gere o ciclo principal da aplicação.
- **controller.c / controller.h** – Responsável pela ligação entre o menu e os módulos funcionais. Gere a lógica de controlo de fluxo do programa.
- **menu.c / menu.h** – Apresenta e gere o menu principal e as interações com o utilizador.
- **display.c / display.h** – Responsável pela apresentação formatada dos dados no terminal, como listagens tabulares e mensagens.
- **terminal_colors.h** – Define macros e constantes para personalização de cores no terminal (ex: ANSI escape codes).
- **ed.c / ed.h** – Implementa a estrutura de dados ED (lista ligada) e as operações sobre a mesma (inserção, remoção, criação de nós, etc).
- **ed_io.c / ed_io.h** – Responsável pelo carregamento e gravação da estrutura ED a partir de ficheiros de texto (input/output).
- **ed_validator.c / ed_validator.h** – Módulo com funções de validação de dados, como verificação de coordenadas duplicadas, frequências inválidas, etc.
- **effect.c / effect.h** – Implementa a lógica de deteção de interferências e geração de efeitos nefastos (caracter #).
- **geometry.c / geometry.h** – Contém funções relacionadas com cálculo de distâncias, alinhamentos e regras geométricas que suportam a lógica dos efeitos nefastos.
- **constants.h** – Define constantes globais utilizadas por vários módulos (ex: distância mínima entre antenas).

Princípios de Clean Code aplicados

- Separação de responsabilidades por ficheiro
- Nomes de funções e variáveis descritivos e em inglês
- Utilização de funções pequenas e coesas
- Eliminação de código duplicado através de funções auxiliares `static`
- Comentários apenas onde o código não for autoexplicativo
- Documentação automática com Doxygen sobre funções públicas

Esta arquitetura permite que o projeto seja facilmente escalado ou adaptado para a Fase 2, onde será necessária a introdução de grafos.

5. Estrutura de Dados Utilizada

Para representar a rede de antenas e os efeitos nefastos, foi definida uma estrutura de dados dinâmica baseada numa **lista ligada simples**. Esta escolha deve-se à necessidade de uma estrutura flexível, que permita inserções e remoções dinâmicas sem a necessidade de reorganização da memória, como seria necessário em arrays tradicionais.

A estrutura principal utilizada é a seguinte:

```
typedef struct {  
    char frequency; // Frequência da antena ou efeito nefasto  
    int x;           // Coordenada X  
    int y;           // Coordenada Y  
    struct ed *next; // Próximo elemento da lista  
} ED;
```

Cada nó da lista representa uma antena ou uma localização com efeito nefasto. A distinção entre ambos é feita através do campo `frequency`, onde o valor especial '#' é utilizado para identificar um efeito nefasto.

Esta abordagem permite manter uma única estrutura comum para representar elementos de diferentes tipos (antenas e interferências), simplificando a lógica de gestão da lista.

Para além da estrutura base, foram implementadas funções auxiliares para criação de nós, inserção ordenada ou em posições específicas, verificação de duplicação de coordenadas e comparação de frequências.

A lista ligada é manipulada através de ponteiros, permitindo a sua expansão ou redução consoante as operações efetuadas, e garantindo uma gestão eficiente da memória.

6. Funcionalidades (TODO...)

7. Testes (TODO...)

8. Conclusão (TODO...)